

모델기반 RL과 몬테카를로 트리 탐색

Dyna-Q 재조명과 “공짜 모델”의 두 축 · MCTS 네 단계와 님 실습 · 멀티에이전트
동역학(선택) · World Models의 상상

Machine Learning 2 · 15주차

한국과학영재학교 (KSA)

Chapter 15

이번 주차 개요

이 챕터를 마치면

- ① 모델기반 RL을 “모델이 어디에서 오느냐”(known/learned)와 “모델을 무엇에 쓰느냐”(학습/계획)의 두 축으로 정리하고, 가치반복 · Dyna-Q · MCTS · MPC가 표의 어디에 있는지 설명할 수 있다.
 - ② MCTS의 네 단계(선택·확장·시뮬레이션·역전파)가 한 번의 시뮬레이션에서 각각 무슨 일을 하는지 설명하고, 님 게임에서 직접 구현해 시뮬레이션 횟수와 탐색 상수 c 의 영향을 실험으로 확인한다.
 - ③ 선택 규칙 **UCT**가 Ch. 2.2의 UCB를 트리 탐색에 적용한 것임을 설명하고, AlphaGo/AlphaZero가 MCTS에 신경망을 결합한 구조를 짚는다.
 - ④ (선택) 여러 에이전트가 학습하면 **비정상성** 문제로 단일 에이전트의 수렴 보장이 무너지는 이유와, 알고리즘이 같아도 **보상 구조**에 따라 결과가 달라지는 이유를 설명한다.
- **15.1** Dyna-Q 재조명: known/learned $\times$ 학습/계획, “공짜 모델”
 - **15.2** MCTS: 네 단계 + UCT, 님 실습, 버그 투어, AlphaGo/AlphaZero
 - **15.3** (선택) 멀티에이전트 RL: 비정상성, 죄수의 딜레마·조율·바위-가위-보
 - **15.4** World Models: 학습된 모델 안에서 상상하며 계획하기

이 장의 주제 = “부분 계획”: 전체 상태 공간을 계산하는 대신, 지금 필요한 갈림길만 깊게 계산한다.

15.1 모델기반 RL 개관: Dyna-Q 재조명과 “공짜 모델”

AlphaGo에서 MCTS로

- 2016년: 딥마인드 **AlphaGo**가 세계 최정상급 프로 이세돌을 꺾음
- 바둑판의 국면 수는 **우주의 원자 수보다 많다고** 알려져 있어서 “가치 근사”(DQN, Ch. 9)만으로는 부족
- AlphaGo의 핵심 무기 중 하나 = **몬테카를로 트리 탐색 (Monte Carlo Tree Search, MCTS)**
- Ch. 7.3에서 맛본 “모델을 활용한 계획”을, 명확한 규칙이 있는 게임에서 훨씬 강력하게 확장한 형태

이 장의 위치

지금까지의 ML2(DQN · PPO · 모방학습)는 대부분 **model-free**: 시행착오로 배움. 이번 장은 그 반대편 — **환경의 규칙이 주어졌을 때 그 모델을 이용해 미리 계획하고 결정하는 방법**. 이 두 축이 합쳐져야 AlphaGo/AlphaZero가 완성된다.

Dyna-Q 복습: 학습된 모델로 가상 경험 만들기

Ch. 7.3의 **Dyna-Q**는 세 가지를 **동시에** 했다:

- ① 실제 환경과 상호작용해 **경험**을 쌓는다
- ② 그 경험으로 간단한 모델 $\hat{P}(s'|s, a)$, $\hat{R}(s, a)$ 를 **학습**한다
- ③ 그 모델로 **가상의 추가 경험**을 만들어 Q값을 더 많이 갱신한다

모델기반 강화학습(Model-Based RL)

Dyna-Q 아이디어의 일반화 — 환경의 모델(전이확률 + 보상함수, 또는 그 근사)을 학습하거나 이미 알고 있다면, “실제로 행동해보지 않고도” 여러 수를 미리 시뮬레이션할 수 있다.

- Dyna-Q는 이 시뮬레이션을 **값 갱신**에 썼다
- 이번 장의 MCTS는 **지금 당장 어떤 행동을 할지 결정하는 데** 직접 쓴다 — 이 점이 차이

“모델을 안다”의 두 갈래

Dyna-Q의 모델 $\hat{P}$, $\hat{R}$ 은 **경험에서 쌓인** 것이었다 (각 (s, a) 마다 “마지막으로 관찰된 (r, s') ”을 기억하는 표).

known model(알려진 모델)

- 바둑·체스: 모델은 학습된 게 아니라 **규칙 자체** — 게임 규칙을 코드로 옮긴 것
- 데이터로부터 추정한 근사가 아니므로 **오차가 전혀 없는 완벽한 모델**

learned model(학습된 모델)

- Dyna-Q처럼 **데이터로부터 학습한** 모델
- 로봇 같은 실제 환경은 보통 이쪽 — 모델이 데이터로부터 배워야 하거나(또는 시뮬레이터로 만들거나) 아예 만들지 못한다

모델이 어디에서 오느냐(known/learned)와 모델을 무엇에 쓰느냐(학습/계획)는 두 개의 독립된 축이다.

두 축으로 모으기

- **학습**(값 개선)에 쓰면 — Dyna-Q의 전형: 모델로 만든 가상 경험을 Q 갱신에 쓸 뿐, “지금 이 수에서 무엇을 할지”를 직접 계산하지 않는다
- **계획**(결정)에 쓰면 — 15.2절 MCTS의 전형: (가상의) 행동 시나리오를 전개해 **지금 당장** 둘 수를 골라낸다

	known model (규칙이 주어짐)	learned model (데이터로 학습)
학습(값 개선)	가치반복·정책반복 (Ch. 4)	Dyna-Q (Ch. 7.3)
계획(지금의 결정)	MCTS·AlphaZero (15.2)	로봇의 MPC 등

- 바둑 = **위쪽줄 오른쪽칸**: 모델은 정확히 알지만 상태 공간이 너무 커서 “전체 계산”(Ch. 4)이 불가능 ⇒ “지금 필요한 것만 계산”(MCTS)으로 방향 전환
- **MPC(Model Predictive Control)**: 매 스텝마다 모델로 후보 행동 시퀀스를 미래로 시뮬레이션해, 결과를 가장 좋게 하는 시퀀스의 **첫 행동**만 취하는 제어 — 산업용 로봇에 널리 쓰이는 “모델로 계획하는” 제어기

역사적 맥락: 따로 자라온 두 계보

- MCTS의 계보는 학습 알고리즘 계보와 **따로** 자라왔다
- 2006년 Kocsis와 Szepesvari가 바둑 탐색에 제안한 것이 출발점, 그 선택 규칙 **UCT**는 **Ch. 2.2의 UCB 밴딧에서 직접 가져온 것** — “학습” 문헌의 도구를 “게임 AI”에 접목한 기술
- AlphaGo(2016)·AlphaZero(2017)가 여기에 **신경망을 엮어** 바둑·체스·시구를 석권한 뒤, 이 일대가 ML 공통 언어로 “모델기반 RL”이라는 이름 아래 묶이기 시작

“모델기반”은 우산 용어

고정된 '학파'가 아니다 — 기준은 “**모델을 이용해 실제 환경과의 상호작용을 줄이는가**”. 모델이 known/learned인지, 용도가 학습/계획인지는 그 안의 독립된 축.

14장의 시뮬레이터(MuJoCo·Isaac Sim)는 known model이 **주어진** 사례 — 13~14장에서 “시뮬레이터 위에서 배우며 다닌”経験を “모델로 계획한다”는 관점에서 다시 짚는 장이다.

모델이 “공짜”인 경우: 체스와 바둑

체스·바둑 같은 게임에는 특별한 사정이 있다 —

- 규칙이 **완벽하게 알려져** 있다
- “이 수를 두면 판이 정확히 이렇게 바뀐다”는 전이확률 $P(s'|s, a)$ 를 Dyna-Q처럼 데이터로부터 배울 필요가 **전혀 없음** (Ch. 4 모델 기반 방법의 이상적 조건)
- 문제는 **상태 수가 너무 많다**: 바둑판의 가능한 국면 수는 약 2×10^{170} 으로 추정

Ch. 4.3 가치반복 같은 “전체 계산”

모든 상태를 한 번씩 순회하며 계산 — 10^{170} 개 상태에서는 근본적으로 불가능.

MCTS(15.2)의 “지금 근처만 깊이”

“지금 당장 놓인 상황 근처만” 깊이 파고드는 탐색이 필요해진다 — 15.2절 MCTS가 풀려는 문제.

실습: 완전탐색이 가능한 경우(틱택토)

“모델을 안다”고 해서 항상 완전탐색(minimax)을 쓸 수 있는 건 아니다 — 상태 수가 관건. 틸만 같고 상태 수가 훨씬 작은 틱택토(3×3)에서 검증:

```
WIN_LINES = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
```

```
nodes_visited = 0
```

```
def winner(board):
```

```
    for a, b, c in WIN_LINES:
```

```
        if board[a] != 0 and board[a] == board[b] == board[c]:
```

```
            return board[a]
```

```
    return 0
```

```
def minimax(board, player):
```

```
    global nodes_visited
```

```
    nodes_visited += 1
```

```
    w = winner(board)
```

```
    if w != 0: return w
```

```
    if all(x != 0 for x in board): return 0 # 무승부
```

```
    scores = []
```

```
    for i in range(9):
```

```
        if board[i] == 0:
```

```
            board[i] = player
```

```
            scores.append(minimax(board, -player))
```

```
            board[i] = 0
```

Made by Sumin Han • suminhan@ksa.hs.kr

결과: 549,946개 vs 10^{165} 배 차이

완전탐색결과선공승

(1=, 무승부0=, 후공승-1=): 0탐색한전체국면수

: 549946

- 두 선수가 모두 최선을 다하면 틱택토는 **항상 무승부**로 끝난다
- 이 결론을 얻는 데 필요한 국면 수는 **549,946개** — 평범한 노트북에서 **1초도 안 걸려** 전부 훑을 수 있다
- 반면 바둑은 이보다 10^{165} 배 이상 — 완전탐색이라는 “정직한 방법” 자체가 **아예 성립하지 않는 규모**

이 격차가 곧 MCTS의 이유

전체 트리를 다 보는 대신, **유망해 보이는** 가지만 **표본추출(sampling)**로 골라 깊이 파고든다 — 15.2절의 MCTS.

손으로 계획 한 스텝: 알려진 모델에 가치반복

모델이 완벽히 알려져 있을 때 “계획”은 어떤 모습인가 — 가장 단순한 형태인 **가치반복**(Ch. 4.3)을 손으로 들여다본다.

- 3상태 예제: $S(\text{출발}) \rightarrow X(\text{중간}) \rightarrow T(\text{목표, 종료})$, 각 상태에서 행동 하나씩, 전이는 결정론적
 $S \xrightarrow{a} X$ (보상 0), $X \xrightarrow{a} T$ (보상 +1), $\gamma = 0.9$

참값 (거꾸로 대입)

$Q(T) = 0, Q(X) = 1 + 0.9 \cdot 0 = 1$,
 $Q(S) = 0 + 0.9 \cdot 1 = 0.9$ — 손에 바로 나온다.

가치반복 한 반복

모든 (s, a) 에 대해 $Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$.
주의: 동기화(synchronous) 갱신 — 새 값을 오래된 Q 로 계산한 뒤 한꺼번에 바꿔 넣는다.

가치반복의 반복 1→3: 값이 “한 칸씩” 퍼져 나간다

반복	계산	결과
1 (초기 Q 전부 0)	$Q(X) \leftarrow 1 + 0.9 \cdot Q(T), Q(S) \leftarrow 0 + 0.9 \cdot Q(X)$	$Q(X) = 1, Q(S) = 0$ (아직 $Q(X)$ 가 퍼지지 않음)
2	$Q(S) \leftarrow 0 + 0.9 \cdot Q(X) = 0.9$	$Q(S) = 0.9, Q(X)$ 는 변하지 않음
3	아무 값도 변하지 않음	수렴

- $Q(S)$ 가 0에서 0.9로 도약하는 데 **정확히 2번**의 반복이 걸렸다
- 반복 1: 목표에서 **1칸** 떨어진 X 만 “+1이 2칸 뒤에 있다”는 사실을 알게 됐고, 2칸 떨어진 S 는 아직 0
- 반복 2: 값이 옛지 $S \rightarrow X$ 를 타고 **한 칸 뒤로 퍼져** S 까지 도착

본질: 한 반복당 한 칸

값 정보가 모델 그래프를 따라 **정확히 한 상태만큼** 퍼져 나간다 — 목표에서 d 칸 떨어진 상태는 정확히 d 번 반복, L 개 링크 체인의 전체 수렴은 $L + 1$ **번째 반복**. (7.3절 5×5 격자: 최단 경로 8스텝 $\Rightarrow$ 9번 반복에 수렴.)

깊이 시뮬레이션 하나가 가치를 “압축”한다

“한 번에 한 칸”을 다른 관점에서 보자 — 우리는 **모델을 안다**(규칙을 안다)고 하니, 체인을 **한 번에 전부 시뮬레이션**할 수 있다:

$$S \xrightarrow{a} X \xrightarrow{a} T$$

- 두 스텝만 시뮬레이션하면 보상 +1을 **바로** 본다
- 가치반복이 2번 반복에 걸쳐 발견한 것과 **동일한 정보를, 깊이 있는 시뮬레이션 하나가 압축해** 준다

MCTS와 이 대응

MCTS의 “시뮬레이션(끝까지 전개) + 역전파(결과를 지나온 노드로 전달)”는 정확히 이 “깊이 시뮬레이션 + 결과 뒤로 전파”의 **표본추출 버전** — 전체 체인이 아니라 “유망해 보이는” 체인을, 전체 상태를 대신 샘플링으로 골라 처리한다는 것만 다를 뿐. 15.2절에서 이 대응을 단계별로 다시 펼쳐 본다.

실습: Q-learning vs Dyna-Q — 가속을 숫자로

위 손계산은 모델이 **완벽히 알려진** 경우. 일반적인 경우는 모델을 **배워야 하는** 경우(Dyna-Q) — 학습된 모델로 계획하는 것이 학습을 실제로 얼마나 가속하는지, 7.3절의 5×5 격자로 재본다.

- 환경: 출발 (4, 0), 목표 (0, 4), 4방향 이동, 이동마다 보상 -1 , 목표 도착 시 0으로 종료, $\gamma = 1$, $\alpha = 0.1$, $\varepsilon = 0.1$
- 참값(손계산): 최단 경로 8스텝 $\Rightarrow$ 최적 리턴 $-7 - Q^*(\text{출발}, \text{위}) = Q^*(\text{출발}, \text{오른쪽}) = -7$, “아래/왼쪽”은 -8
- known model이면 가치반복 9번 반복(계산 900회)으로 끝남 — **계획이 사실상 공짜인** 경우
- learned model(Dyna-Q): 실제 스텝 1회당 모델 엔트리 1개만 늘어나므로 모델 커버리지가 **경험과 함께 서서히 성장**

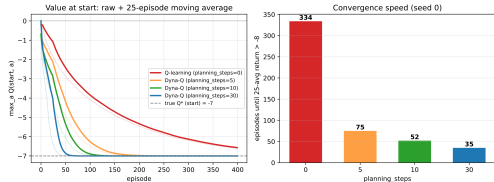
$planning_steps \in \{0, 5, 10, 30\}$ 을 400 에피소드로 비교 (시드 0, 시드 1~3도 유사한 범위).

결과: 334 에피소드 → 35 에피소드

설정	“거의 최적” 최초 진입†	400에피소드 시 max Q(출발)
가치반복 (known model)	9번 반복	정확히 -7.00
Q-learning (<i>planning_steps</i> =0)	334 에피소드	-6.61
Dyna-Q (<i>planning_steps</i> =5)	75	-7.00
Dyna-Q (<i>planning_steps</i> =10)	52	-7.00
Dyna-Q (<i>planning_steps</i> =30)	35	-7.00

† 25에피소드 평균 리턴 > -8

- (1) **가속은 극적**: 실제 상호작용 35회로, 계획 없이는 334회가 걸렸던 학습이 끝남(약 **9.5배**). “실제 스텝 1회 = 로봇 이동 1초” 환경이면 쓸 수 있는 수준까지의 **벽시계가 같은 비율로 줄어듦**
- (2) **한계 효과 감소**: 0→10은 334→52(6.4배) 크게 빨라지나, 10→30은 52→35(1.5배)로만 — 계획 계산량 2배가 수렴 2배가 아님
- *planning_steps*도 α, ϵ 처럼 **검증 성능으로 튜닝하는 하이퍼파라미터**



위: max Q(출발) 학습 곡선 — 0(빨강)이 에피소드 330쯤 -7 도달, 10/30(초록/파랑)은 40~50. 회색 점선 = 참값 -7. 아래: “거의 최적” 최초 진입 에피소드 수.

루프의 대응: Dyna-Q와 AlphaZero는 같은 구조

마지막 점이 이 장의 줄거리를 잇는다:

두 루프는 동일한 구조

Dyna-Q 루프(learned model $\leftrightarrow$ 계획으로 값 개선)와 **AlphaZero 루프**(신경망 $\leftrightarrow$ MCTS 탐색)은 **동일한 구조** — “모델” 자리에 학습된 신경망, “Q 테이블” 자리에 가치 헤드, “계획 갱신” 자리에 MCTS 탐색이 있을 뿐.

- 7.3절의 루프 = “모델로 Q를 개선하는 **학습**”
- 15.2절의 MCTS = “모델로 지금의 결정을 계산하는 **계획**”
- 즉, **같은 두 요소 루프를 학습/계획이라는 두 축에서 다시 보는 셈**

모델이 known인지 learned인지, 학습에 쓰는지 지금의 결정에 쓰는지 — 두 축의 교차점에 무엇이 있는가?

자주 하는 실수: “모델이 있으니 학습이 필요 없다”

모델기반 RL을 처음 배울 때의 흔한 오해 —

“환경의 규칙을 완벽히 안다면, Ch. 4의 가치반복 같은 planning으로 바로 최적 정책을 계산할 수 있으니 **신경망 학습이 아예 필요 없다**”

- 이건 **상태 수가 작을 때만** 맞는 말
- 바둑처럼 상태 수가 천문학적이면, “모델을 안다”는 것과 “그 모델로 최적 행동을 **실제로 계산**할 수 있다”는 것은 **전혀 다른 얘기** — 그래서 AlphaGo/AlphaZero는 여전히 신경망을 학습시킨다
- 반대로 Dyna-Q처럼 모델이 **학습된** 것일 때는 **모델 오차**가 Q값에 누적: 초기에 몇 안 되는 경험으로 학습한 부정확한 모델로 가상 경험을 너무 많이 만들면, 그 부정확함이 Q값에 스며들어 오히려 **학습을 방해**

기억할 것

“모델기반”이 공짜로 정확한 답을 주는 게 아니다 — **모델의 정확도(또는 완전성)과 상태 공간의 크기 둘 다**가 계획의 실현 가능성을 결정한다.

역방향 실수: 모델이 known하면 학습 불필요?

“모델을 완벽히 안다면(known model), 학습은 전혀 필요 없다”는 역방향 버전 —

- 틱택토 실습: 이 주장은 **상태 공간이 작을 때만** 성립
- 바둑: 상태 공간이 너무 커서 **완벽한 모델이 있어도** 전체 공간으로 최적 행동을 계산할 수 없다

AlphaGo/AlphaZero가 규칙을 완전히 알면서도 신경망을 학습시킨 이유는 “모델을 배우기 위해”가 아니라 “탐색 공간을 압축하기 위해서”.

모델(규칙)이 말하는 것

“이 수를 두면 판이 어떻게 변하는지” — 답이 **어디에** 있는지.

- MCTS는 신경망의 힌트를 따라 유망한 소수 가지만 골라 깊이 시뮬레이션 — 제한된 시간(예: 한 수당 2초)에 수백~수천 번의 시뮬레이션으로 “모델이 아는 답”을 채굴
- 모델을 몰랐다면 시뮬레이션 한 번도 못 돌렸을 텐데, 모델이 있어도 **탐색 방향**을 몰랐다면 시뮬레이션을 엉뚱한 가지만에 흘릴 뻔

신경망이 말하는 것

“어떤 갈림길부터 들여다볼 가치가 있는가” — **어디를** 볼지.

“모델을 안다”는 것은 계획의 **재료**를, “학습”은 계획의 **방향**을 준비해 준다 — 둘은 서로를 대체하지 않는다.

자주 묻는 질문(15.1)

- **Q: Ch. 4 가치반복도 “모델기반”인데, 왜 이 장에서 다시 이 용어?** — 차이는 스케일과 쓰임새: Ch. 4는 상태가 적어 정책 **전체**를 미리 계산, 이번 장(Dyna-Q, MCTS)은 상태가 너무 많아 “지금 이 상태에서 무엇을 할지” **만** 계산
- **Q: 모델기반이 model-free보다 항상 좋은가?** — **아니** — 정확한(또는 잘 학습된) 모델이 있을 때 데이터 효율이 훨씬 좋지만, 모델 학습 비용과 **오차 누적** 위험이 있음. 로봇 팔처럼 물리 법칙이 복잡한 환경은 Ch. 9-13의 model-free 쪽이 더 실용적인 경우 많음
- **Q: known model이면 learned model 얘기를 안 해도 되나?** — 실제 환경 대부분이 바둑처럼 규칙이 완전히 알려진 게임이 **아님**(로봇·화학 공정·교통·인간-기계 상호작용). “공짜 모델”은 예외적 특권, **일반은 learned model** — 2010년대 이후 연구(world model 계열)의 주목적도 이 케이스 (15.4절)

자주 묻는 질문: DQN 경험 재사용과의 관계

Q: known model 경우와 Ch. 9 DQN의 경험 재사용 (experience replay)은 관계가 있나?

둘 다 “손에 있는 정보를 더 많이 쓰자”는 기술이지만 재사용의 원천이 다르다:

경험 재생(experience replay)

- **실제** (s, a, r, s') 튜플을 버퍼에서 다시 뽑아 쓰기 때문에 **모델 오차가 없음**
- 단, **경험하지 않은 전이는 만들 수 없다**
- “**안전하지만 새것을 못 만든다**”

모델 계획(Dyna)

- 모델로 전이를 **생성** — 경험이 커버하지 않은 상태까지 **갈 수 있음**
- 단, **모델 오차를 짊어진다**
- “**새것을 만들 수 있지만 오차가 동반**”

9장 이후의 모델기반 DQN 변형들은 학습된 신경망 모델을 엮어 이 둘을 **결합**하는 방향으로 나아간다.

확인 문제(15.1)

- 1 **체인 전파:** 체인을 $S \rightarrow X_1 \rightarrow \dots \rightarrow X_8 \rightarrow T$ (10개 상태, 링크마다 보상 0, 마지막 링크만 +1, $\gamma = 0.9$)로 늘리면, 가치반복에서 $Q(S)$ 가 0이 아닌 값이 되려면 몇 번 반복이 필요한가? known model로 시뮬레이션만 하면 몇 스텝이면 충분한가? (힌트: “한 번에 한 칸” vs “한 스텝에 한 칸”.)
- 2 **틀린 모델:** dyna_q_grid의 모델 갱신 줄 `model[(s, a)] = (r, ns)`를 `model[(s, a)] = (r, (4 - ns[0], 4 - ns[1]))`로 (다음 상태를 판 대각 반대 자리라 예측하는) 틀린 모델로 바꿔 `planning_steps=30, 400` 에피소드 학습 후 출발점 (4, 0)의 max Q 와 25에피소드 평균 리턴을 기록. “아래/왼쪽” 엔트리가 다음 상태를 **목표** (0, 4)로 예측하는 것을 확인하고, 이 엔트리가 Q 값과 greedy 정책을 어디로 끌어당기는지 설명.
- 3 **도로 vs 목적지:** `planning_steps=0`(순수 Q-learning)도 400 에피소드 후 greedy 정책이 25개 상태 전 Q^* 와 일치(불일치 0/25). 그럼에도 계획이 실제로 “가속”한 것은 무엇인가? (“처음 진입” 열 참고.)
- 4 **루프 대응:** Dyna-Q 루프의 세 요소(학습된 모델, Q 테이블, 계획에 의한 가상 갱신)를 AlphaZero의 구성 요소(신경망 정책/가치 헤드, MCTS)와 짝짓고 각 대응을 한 문장으로.

15.1 핵심 정리

모델기반 RL = 모델을 이용해 **실제 환경과의 상호작용을 줄이는 방법들의 묶음**

모델의 **출처**(known/learned)와 **용도**(값 개선 = 학습 / 지금의 결정 = 계획)이라는 **두 축이 독립적**.

방법	두 축에서의 위치
가치반복	known + 학습
Dyna-Q (Ch. 7.3)	learned + 학습
MCTS	known + 계획
AlphaZero	신경망이 모델의 역할을 맡은 확대판 Dyna 루프

- “모델을 안다”는 것이 계획의 **재료**를, “학습”은 계획의 **방향**을 준비해 준다 — 둘은 서로를 대체하지 않는다

모델기반 RL은 하나의 알고리즘이 아니라 “모델이 실제 상호작용의 일부를 대신할 수 있다”는 한 가지 아이디어로 묶인 **방법들의 가족**. 다음 15.2절: 두 축의 가장 날카로운 교차점 — known model로 “지금의 결정”을 계산하는 **MCTS**.

15.2 몬테카를로 트리 탐색: 이론과 실습

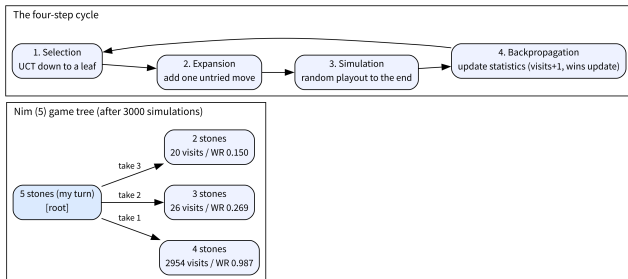
MCTS: 네 단계의 반복

MCTS는 현재 상태에서 시작해, **정해진 횟수**만큼 다음 네 단계를 반복해 “어떤 수가 가장 유망한가”에 대한 **통계**를 쌓는다:

- 1 **선택(Selection)**: 루트(현재 상태)에서, “지금까지의 성과” + “아직 덜 시도해본 정도”를 함께 고려하는 규칙 (Ch. 2.2의 UCB와 **정확히 같은 발상**)으로 자식을 골라 내려가, 아직 다 확장되지 않은 노드까지
- 2 **확장(Expansion)**: 그 노드에 새로운 자식(아직 안 가본 수)을 **하나** 추가
- 3 **시뮬레이션(Simulation/Rollout)**: 그 새 노드에서 (간단한 정책으로, 극단적으로 **완전 무작위**) 끝까지 플레이 — Ch. 5의 몬테카를로처럼 결과 (승/패)를 얻는다
- 4 **역전파(Backpropagation)**: 그 결과를 지나온 모든 노드에 통계로 반영(방문 +1, 승률 갱신)

이 “역전파”는 ML1 Ch. 9의 역전파와 **이름만 같을 뿐 완전히 다른 개념**: 신경망의 그래디언트가 아니라, 트리의 **승패 통계**를 전달한다.

네 단계가 순환하는 구조



- 왼쪽: 님 게임(돌 5개) 게임 트리 — 루트에서 “1개/2개/3개 가져가기” 세 갈림
- 각 자식 노드에 **방문 횟수 + 승률**이 쌓임 (3000회 시뮬레이션 후 실제 값: **2954/0.987**, 26/0.269, 20/0.150)
- 오른쪽: 선택은 트리를 **내려가고**, 역전파는 지나온 **모든** 노드의 통계를 갱신한 뒤 다음 선택으로 돌아온다

- 수백~수만 번 반복한 뒤, 루트에서 **가장 많이 방문된** 자식(반복적으로 가장 유망하다고 확인된 수)을 실제로 둘 수로 선택
- “가장 많이 방문된 것”과 “승률이 가장 높은 것”은 **대개 일치하지만 항상 같지는 않다** (FAQ에서 다룬다)

계보: moa*에서 UCT까지

- “무작위로 끝까지 해보고(시뮬레이션), 그 결과를 트리의 통계로 적재적재하게 나누어 모은다(선택 + 역전파)”는 이 아이디어의 계보는 1990년대 **오톨로 (Othello)** 프로그램의 **moa*** 알고리즘까지 거슬러 올라감

UCT (Upper Confidence bound applied to Trees)

2006년 Kocsis와 Szepesvari가 이 가문의 방법들을 **UCT** 라는 이름으로 정리하며, Ch. 2.2의 **팔레트 아머드 밴딧** 문제를 트리 탐색에 연결하는 **정확한 수식**을 줌 — 이 UCT가 오늘 배울 선택 규칙.

- 이후 MCTS는 바둑·체스·광물자원 게임 등 “경우의 수가 너무 많아 **완전탐색이 불가능한**” 문제들의 **표준 계획 알고리즘**이 됨
- 2016년 AlphaGo의 핵심 구성요소로

하나의 시뮬레이션을 따라가보자

루트(돌 5개, **내 차례**)에서 시작:

- ① **선택** — 루트에 자식이 **하나도 없으므로** 루트에서 즉시 정지
- ② **확장** — 아직 안 가본 수(예: “2개 가져가기”)를 무작위로 하나 골라 자식으로 추가
- ③ **시뮬레이션** — 새 노드(돌 3개, **상대 차례**)부터 무작위 플레이로 게임 끝까지 두어 승자를 판정
- ④ **역전파** — 루트와 그 새 자식, **두 노드 모두**에 “방문 1회”를 더하고, 승자가 “그 노드로 두어 도달한 플레이어”라면 승 1회도 더함

“모든 수를 최소 한 번” 보장은 여기서 온다

첫 3회 시뮬레이션은 세 갈림의 각각을 한 번씩 “열어보는” 데 쓰임 — 방문 0인 자식은 항상 우선되기 때문. 4회차부터 선택 단계의 UCT가 실제로 **작용**하기 시작한다.

UCT: 선택 단계의 규칙

“성과 + 덜 시도해본 정도”는 UCB1의 수식과 **정확히 같은 형태**. 노드 s 에서 자식 a 를 고를 때:

$$\text{UCT}(s, a) = \frac{Q(s, a)}{N(s, a)} + c \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

- $Q(s, a)$ = 자식 a 의 누적 승(수), $N(s, a)$ = 방문 횟수, $N(s)$ = 부모 s 의 방문 횟수
- 첫 항 = **활용**(“이 수의 승률이 얼마나 높은가”), 두 번째 항 = **탐험**(“이 수를 얼마나 덜 보았는가”)

탐험 항의 감소

탐험 항의 크기는 대략 $1/\sqrt{N(s, a)}$ 로 감소 — 한 수가 **4번** 시도되면 탐험 보너스가 **절반**, **100번**이면 **1/10**으로 줄어듦. 그래서 “명백히 낮은 수”는 소수 몇 번 추가 시도 후 **방치**되고, “경합 중인 수들”은 비례해서 계속 분배된다.

UCT 선택 — 코드로

```
import math

def uct_select(node_children, parent_visits, c=1.4):
    # node_children: {move: (wins, visits)}
    # 과UCB1 정확히같은형태 -- Chapter 2.2 그대로게임트리에적용한것
    return max(node_children, key=lambda m: node_children[m][0] / node_children[m][1] +
               c * math.sqrt(math.log(parent_visits) / node_children[m][1]))
```

숫자로 체감 — 부모 **16회** 방문, 자식 세 개 (A: 승 7/방문 10, B: 승 3/방문 5, C: 승 0/방문 1), $c = 1.4$ 일 때:

- A: $7/10 + 1.4\sqrt{\ln 16/10} = 0.70 + 0.27 = \mathbf{1.437}$
- B: $3/5 + 1.4\sqrt{\ln 16/5} = 0.60 + 0.59 = \mathbf{1.643}$
- C: $0/1 + 1.4\sqrt{\ln 16/1} = 0.00 + 1.44 \approx \mathbf{2.331}$

승률 0인 C가 선택된다.

왜 승률 0인 C가 선택되는가

“지금까지 한번도 이겨본 적 없다”는 수인데 왜 C인가:

- 아직 1번밖에 시도하지 않았으므로, “진짜 나쁜 수”일 가능성과 “아직 제대로 평가되지 못한 수”일 가능성을 분간할 수 없음

C를 몇 번 더 보내면 —

- **진짜 나쁘다면**: 승률이 확인되고 탐험 향이 줄어 자연스럽게 방치
- **운 좋게 괜찮다면**: 활용 향이 올라와 본격적 승부수가 됨

Ch. 2.2의 트레이드오프

탐험-활용 트레이드오프가 트리 탐색에서 그대로 작동하는 모습. (15.3절 연습문제 1이 바로 이 계산을 빈칸으로 준다.)

MCTS 전체: 게임 규칙 + 노드 (님)

게임 = 님(Nim): 돌을 1~3개씩 번갈아 가져가고 **마지막 돌을 가져가는 쪽이 이김**. 노드는 그 노드에서 “이동한 플레이어”(mover)의 관점의 통계를 담는다는 점에 주목 — 역전파가 무엇을 어떻게 더하는지가 전부 여기서 결정된다:

```
import math
import random

def nim_moves(state):
    """이 상태에서가능한수 : 남은돌수만큼 , 개개개1/2/3 가져가기."""
    return list(range(1, min(3, state) + 1))

def nim_step(state, move, player):
    """개를move 가져간다: 남은돌이줄고 , 상대차례가된다 ."""
    return max(0, state - move), -player

def nim_terminal(state):
    return state == 0

def nim_rollout(state, player, rng):
    """시뮬레이션롤아웃(): 완전히무작위정책으로끝까지 . 승자마지막
    ( 돌을가져간플레이어 ) 반환. 이미종료된상태면막이동한쪽이승자
    """
    if nim_terminal(state):
```

Made by Sumin Han · suminhan@ksa.hs.kr

MCTS 전체: 4단계 조립

```
def mcts_nim(stones, iters, c=1.4, seed=0):
    """ 돌이 개인 stones 님게임에서 루트 위치의 MCTS. 루트 노드 반환 """
    rng = random.Random(seed)
    root = MCTSNode(stones, player=1, parent=None)
    for _ in range(iters):
        # 1) 선택: 아직 다 확장되지 않은 리프까지 로 UCT 내려간다.
        node = root
        while not nim_terminal(node.state) and len(node.children) == len(nim_moves(node.state)):
            node = max(node.children.values(), key=lambda ch:
                ch.wins / ch.visits + c * math.sqrt(math.log(node.visits) / ch.visits)
                if ch.visits else float('inf'))
        # 2) 확장: 안가본 수 하나를 자식으로 추가. 이미( 끝난 상태면 확장 생략 )
        if not nim_terminal(node.state):
            unexpanded = [m for m in nim_moves(node.state) if m not in node.children]
            m = rng.choice(unexpanded)
            next_state, next_player = nim_step(node.state, m, node.player)
            node.children[m] = MCTSNode(next_state, next_player, parent=node)
            leaf = node.children[m]
        else:
            leaf = node
        # 3) 시뮬레이션: 리프에서 무작위 정책으로 끝까지, 승자 판정
        winner = nim_rollout(leaf.state, leaf.player, rng)
        # 4) 역전파: 지나온 모든 노드 방문 by Sumin Han, 승자가면 승자 수 +1
        cur = leaf
```

세 자잘한(그러나 치명적인) 지점 ①: 선택의 정지 조건

- “아직 다 확장되지 않은 리프까지” — while 조건이 `len(node.children) == len(nim_moves(state))`인 이유: 자식이 **일부만** 있는(부분 확장) 노드에서 UCT로 계속 내려가면 안 된다
- 아직 한 번도 열어보지 않은 자식이 남아 있으면, **우선 그걸 열어야**(확장의 “모든 수를 최소 한 번” 보장) 트리의 통계가 **공정**해진다

이 조건을 `while node.children:`으로 잘못 쓰면

모든 방문이 **첫 번째로 열린 자식 하나**에 집중되는 버그가 나온다 — 아래 “버그 투어”에서 실증.

세 지점 ②: 역전파의 관점 “mover”에게 승을

- **mover** = “이 노드로 이동한 플레이어”
- 자식 노드라면 **부모의 차례인** 플레이어 (`cur.parent.player`), 루트라면 이동한 플레이어가 없으므로(`parent is None`) 현재 차례 플레이어(`cur.player`)를 **대신** 쓰되 — 루트 자체의 통계는 자식 선택에 쓰이지 않으므로 ($\ln N(s)$ 에 $N(s)$ 만 들어가) **무해**

이 한 줄이 틀리면 — “차례인 플레이어”에게 승을 더하면

UCT가 상대에게 유리한 수를 좋은 수로 착각하게 된다. 왜 그런지는 버그 투어에서 숫자로 본다.

세 지점 ③: 터미널 상태의 확장

- 게임이 이미 끝난 노드(돌 0개)에서는 nim_moves가 **빈 리스트**를 돌려주므로, 터미널 검사를 하지 않고 확장을 시도하면 IndexError
- nim_rollout도 **같은 이유**로 첫 줄에 터미널 처리가 있다

“마지막 돌을 가져간 쪽이 이긴다”를 코드로

“누가 마지막에 이동했는가”를 -player(차례가 전달되기 전의 플레이어)로 판정해야 한다는 점에 주의 — nim_rollout이 종료 상태에서 -player를 반환하는 이유.